

C++ Assignment 2 Theory Questions

Q1.2

1. Why must initialization happen before assignment?

Initialization happens when a variable or object is created and given its **first value**. Assignment happens **after the variable already exists**.

When an object is created, memory is allocated for it. At that moment, it is better to initialize the variables so they start with **valid values**. If we skip initialization, variables may contain **garbage values**, which can cause unexpected behavior.

Therefore, initialization ensures that an object starts in a **proper and predictable state**.

2. When is initialization preferred over assignment?

Initialization is preferred in the following situations:

1. **Inside constructors** when creating objects.
2. **For const variables**, because their value cannot be changed after initialization.
3. **For reference variables**, because they must be bound to a variable when they are created.
4. **For better performance**, since initialization directly sets the value instead of creating the variable first and then assigning it.

Thus, initialization is generally **safer and more efficient than assignment**.

Q2.2

How reference variables act as aliases and how that affects swapByReference()

A **reference variable** in C++ acts as an **alias (another name)** for an existing variable.

For example:

```
int a = 10;  
int &ref = a;
```

Here, ref is not a new variable. It is simply another name for a. Any change made through ref will directly affect a.

When we use **swapByReference(int &a, int &b)**:

- The function parameters refer directly to the original variables.
- No copies are created.
- The values are swapped in the original memory locations.

Because of this, the changes made inside the function **actually modify the original variables**, which is why swapping works correctly with references.

Q3.2

1. Two differences between pointer and reference

Pointer	Reference
A pointer stores the memory address of another variable.	A reference is an alias (another name) for an existing variable.
A pointer can be reassigned to point to another variable.	A reference cannot be changed once it is initialized.

2. Why references cannot be reseated but pointers can

A pointer is a variable that stores an address. Because it is a normal variable, we can change the address it stores.

Example:

```
int a = 10;
int b = 20;
int *ptr = &a;
ptr = &b;
```

Here the pointer now points to a different variable.

However, a reference must always refer to the **same variable it was initialized with**. Once a reference is created, it cannot be made to refer to another variable. This is why references cannot be reseated.

3. Why references cannot be NULL

A reference must always refer to a **valid existing variable**.

Unlike pointers, references do not store addresses explicitly. Instead, they behave like the original variable itself. Because of this design, the C++ language does not allow references to be **NULL or uninitialized**.

Pointers, on the other hand, can store a NULL value because they simply store addresses.

Q4.3

1. Why constructors are not called when using malloc

malloc() is a function from the **C programming language**. It only allocates raw memory and does not know anything about C++ classes or objects.

Since constructors are part of **C++ object initialization**, they are not automatically called when memory is allocated using malloc().

2. Why new is preferred in C++

The new operator is designed specifically for C++. It provides several advantages:

- It automatically calls **constructors** for objects.
- It returns a **properly typed pointer**.
- It integrates well with C++ features like classes and objects.

Because of these advantages, new is considered **safer and more suitable for C++ programs**.

3. Difference in return types and initialization

new	malloc
Returns a pointer of the correct data type	Returns void* pointer
Automatically calls constructors	Does not call constructors
Can initialize objects	Only allocates raw memory

Example:

```
int *p = new int;
```

vs

```
int *p = (int*) malloc(sizeof(int));
```

4. Why malloc cannot initialize complex types

Complex types like **classes and objects** often require constructors to initialize their data members.

Since malloc() only allocates raw memory and does not call constructors, it cannot properly initialize objects that require constructor logic. This is why it is not suitable for complex C++ types.

Q5.2

1. When does the compiler generate a default constructor?

The compiler automatically generates a **default constructor** when:

- No constructor is defined by the programmer in the class.

This default constructor simply creates the object but does not initialize members with meaningful values.

2. When does the compiler NOT generate a default constructor?

The compiler does not generate a default constructor if the programmer has already defined **any constructor** in the class.

For example:

```
class Student {  
public:  
    Student(int r);  
};
```

In this case, the compiler will not create a default constructor automatically.

3. Can constructors be overloaded?

Yes, constructors can be overloaded in C++.

Constructor overloading means defining **multiple constructors in the same class with different parameter lists**.

Example:

```
class Student {  
public:  
    Student();  
    Student(int r);  
    Student(int r, string n);  
};
```

Each constructor will be called depending on the arguments used while creating the object.

Q6.3

1. Why must const members be initialized in the initializer list?

A const member variable cannot be modified after it is created. Therefore, it must be given a value **at the time of initialization**.

Initializer lists allow us to initialize such variables directly before the constructor body executes.

Example:

```
Employee(int id) : employeeld(id) {}
```

2. What happens if you try to assign the value of a const member inside the constructor body?

If we try to assign a value to a const member inside the constructor body, the compiler will generate an **error**.

Example:

```
employeeId = id;
```

This fails because const variables cannot be modified after initialization.

3. Why is initializer list faster than assignment?

Initializer lists are faster because:

- They **directly initialize** member variables when the object is created.
- Assignment requires **two steps**: first the variable is created, and then a value is assigned.

Therefore, initializer lists improve **efficiency and performance**, especially when dealing with complex objects.